AZ-400

Azure DevOps Engineer Expert Interview Questions & Answers

Comprehensive Preparation Guide

300 Q&A | **10 Sections** | **All Levels** | **Exam-Ready**

Compiled by Akhilesh Chaudhari | TechShiksha

Introduction

About This Guide

This guide provides 300 carefully curated interview questions and answers covering all major topics tested in the **Microsoft Azure DevOps Engineer Expert (AZ-400)** certification exam and senior DevOps engineering interviews. Questions span ten core domains with a balanced mix of Beginner, Intermediate, and Advanced difficulty levels.

Exam Domain Coverage

AZ-400 covers: Configure processes and communications (10-15%), Design and implement source control (15-20%), Design and implement build and release pipelines (40-45%), Develop a security and compliance plan (10-15%), and Implement an instrumentation strategy (10-15%). This guide covers all these areas comprehensively across 10 focused sections.

How to Use This Guide

Work section by section or focus on your weak areas. **Difficulty pills** (Beginner / Intermediate / Advanced) help you calibrate depth. Pay special attention to Section 9 (Scenarios) and Section 10 (Troubleshooting) — these closely mirror real interview formats for senior DevOps roles. Supplement this guide with hands-on Azure DevOps and Azure portal practice.

Difficulty Levels

Beginner — Foundational concepts expected of any DevOps practitioner. **Intermediate** — Implementation-level knowledge for experienced engineers. **Advanced** — Architecture, design trade-offs, and complex scenarios for senior/expert roles requiring deep practical experience.

Table of Contents

Difficulty Legend

BEGINNER Foundational concepts

TE Implementation-level knowledge

ADVANCED Architecture & design

Section 1: Azure DevOps Fundamentals

Azure DevOps is Microsoft's end-to-end DevOps platform providing Boards, Repos, Pipelines, Test Plans, and Artifacts. These questions cover core concepts, service relationships, and organizational setup essential for every AZ-400 candidate.

Q1. BEGINNER

What is Azure DevOps and what are its five core services?

Answer:

Azure DevOps is a cloud-based DevOps platform by Microsoft. Its five core services are: **Azure Boards** (work item tracking, Agile planning), **Azure Repos** (Git or TFVC source control), **Azure Pipelines** (CI/CD automation), **Azure Test Plans** (manual and exploratory testing), and **Azure Artifacts** (package management for NuGet, npm, Maven, PyPI).

Q2. BEGINNER

What is the difference between an Azure DevOps Organization and a Project?

Answer:

An **Organization** is the top-level container that groups related projects, manages billing, and controls global policies. A **Project** lives inside an organization and contains all DevOps services (Boards, Repos, Pipelines, etc.) for a specific product or team. One organization can host multiple projects, each with independent settings and permissions.

Q3. BEGINNER

What are Azure DevOps Service Connections?

Answer:

Service Connections are named, secured credential stores that allow pipelines to authenticate to external services such as Azure subscriptions, GitHub, Docker registries, Kubernetes clusters, and SonarQube. They abstract credentials away from pipeline YAML, can be scoped to specific pipelines, and support managed identities, service principals, and personal access tokens.

Q4. BEGINNER

What is the difference between Classic and YAML pipelines in Azure DevOps?

Answer:

Classic pipelines use a visual GUI designer stored in Azure DevOps — easy to create but less portable and harder to review in code. **YAML pipelines** are defined as code in the repository, enabling version control, pull request reviews, and reuse via templates. YAML pipelines are the recommended approach for all new CI/CD implementations.

Q5. TE

What are Environments in Azure Pipelines?

Answer:

Environments are collections of resources (Kubernetes namespaces, Azure App Services, VMs) representing deployment targets like Dev, QA, or Production. They provide **deployment history**, **approval gates**, **branch control checks**, and **audit trails** for every deployment, enabling governance and rollback visibility at each stage.

Q6. BEGINNER

What is an Agent Pool in Azure Pipelines?

Answer:

An Agent Pool is a collection of build/release agents that execute pipeline jobs. **Microsoft-hosted pools** provide fresh VMs per job with pre-installed tools. **Self-hosted pools** use your own machines for custom software, network access, or performance requirements. Agents within a pool run one job at a time by default.

Q7. BEGINNER

What is the purpose of Azure Boards in a DevOps workflow?

Answer:

Azure Boards provides Agile project management tools including **Work Items** (Epics, Features, User Stories, Tasks, Bugs), **Kanban Boards**, **Sprints**, **Backlogs**, and **Delivery Plans**. It links code commits, pull requests, and builds to work items, providing traceability from requirement to deployment.

Q8. TE

What are Variable Groups in Azure Pipelines?

Answer:

Variable Groups store a set of related variables shareable across multiple pipelines within a project. They can be linked to **Azure Key Vault** to dynamically retrieve secrets at runtime, avoiding hardcoded credentials. Variable groups support RBAC to control who can read or modify sensitive pipeline variables.

Q9. TE

What is a Pipeline Template in Azure DevOps?

Answer:

Pipeline Templates are reusable YAML files defining steps, jobs, or stages that can be referenced from other pipelines. They promote **DRY (Don't Repeat Yourself)** principles, enforce organizational standards, and reduce duplication. Templates can be stored in the same repo or a centralized template repository shared across the organization.

Q10. TE

How do you control pipeline execution order and dependencies?

Answer:

Use the **dependsOn** keyword to declare dependencies between stages or jobs. Use **condition** expressions to run stages when previous stages succeed, fail, or meet custom conditions. **Gates and approvals** on Environments add human or automated checks. Work within a job is sequential by default; parallel jobs run concurrently when defined without dependencies.

Q11. TE

What is the Azure DevOps REST API used for?

Answer:

The REST API enables programmatic interaction with all Azure DevOps services — creating work items, triggering pipelines, querying build results, managing repositories, and configuring settings. It is used for automation scripts, custom dashboards, third-party integrations, and extending Azure DevOps capabilities beyond the built-in UI.

Q12. TE

What are Deployment Groups in Azure DevOps?

Answer:

Deployment Groups are sets of target machines (VMs, physical servers) registered with Azure DevOps for **agent-based deployments** using Classic release pipelines. A Deployment Group Agent is installed on each target machine, enabling rolling deployments, tags-based targeting, and parallel deployments across server farms.

Q13. TE

What is the difference between Build Artifact and Pipeline Artifact?

Answer:

Build Artifacts (PublishBuildArtifacts task) upload files to Azure DevOps storage associated with the build. **Pipeline Artifacts** (PublishPipelineArtifact task) use a faster upload mechanism optimized for large files and are preferred for YAML pipelines. Pipeline Artifacts also support file deduplication across runs.

Q14. ADVANCED

How does Azure DevOps support multi-team scaling?

Answer:

Azure DevOps scales for multiple teams through: **Team-level Boards and Backlogs** scoped to team area paths, **Delivery Plans** for cross-team sprint visibility, **inherited process templates** for consistent work item types, **organization-level policies** for governance, and **multi-stage YAML pipelines** with template libraries shared via centralized repos.

Q15.

ADVANCED

What is the Azure DevOps Audit Log?

Answer:

The Audit Log records security and administrative events across the organization — user additions/removals, permission changes, policy modifications, pipeline approvals, and service connection changes. Logs can be exported to Azure Monitor, Splunk, or other SIEM tools for compliance monitoring and incident investigation.

Section 2: CI/CD Pipelines

CI/CD pipeline design is the core of the AZ-400 exam. This section covers pipeline stages, triggers, artifact management, deployment strategies, and advanced YAML concepts for building production-grade automation workflows.

Q1. BEGINNER

What is Continuous Integration (CI) and why is it important?

Answer:

Continuous Integration automatically builds and tests code every time a developer commits changes to a shared repository. CI catches integration bugs early, provides fast feedback, enforces code quality gates (linting, unit tests, security scans), and ensures the main branch is always in a releasable state. Azure Pipelines implements CI via trigger-based builds.

Q2. BEGINNER

What is the difference between Continuous Delivery and Continuous Deployment?

Answer:

Continuous Delivery ensures every successful build is automatically prepared and validated for production, but requires a manual approval to actually deploy. **Continuous Deployment** automatically deploys every passing build to production without human intervention. AZ-400 designs implement CD using deployment stages with environment approvals to balance speed and control.

Q3. TE

How do you configure multi-stage YAML pipelines in Azure DevOps?

Answer:

Multi-stage YAML pipelines use the **stages** keyword. Each stage contains jobs; each job contains steps. Example: stages □ Build, Test, Deploy-Dev, Deploy-Prod. Use **dependsOn** to sequence stages, **condition** to run stages conditionally, and **environment** in deployment jobs to trigger approval gates and record deployment history.

Q4. TE

What are pipeline triggers and how do you configure them?

Answer:

Pipeline triggers control when pipelines run automatically: **CI triggers** (on push to specific branches), **PR triggers** (on pull request creation/update), **Scheduled triggers** (cron expressions), and **Pipeline triggers** (on completion of another pipeline). Configure using the **trigger**, **pr**, **schedules**, and **resources.pipelines** keys in YAML.

Q5. ADVANCED

What deployment strategies does Azure Pipelines support?

Answer:

Azure Pipelines supports via the **strategy** key in deployment jobs: **runOnce** (single deployment, simplest), **rolling** (deploys to a subset of targets at a time), **canary** (deploys to a small percentage first, validates, then completes rollout). Each strategy supports **preDeploy**, **deploy**, **routeTraffic**, and **postRouteTraffic** lifecycle hooks.

Q6. ADVANCED

How do you implement blue-green deployments using Azure Pipelines?

Answer:

Blue-green with App Service: (1) Deploy new version to the **staging slot**. (2) Run smoke tests against staging slot URL. (3) Use the **AzureAppServiceManage** task to perform a slot swap, making staging the new production. (4) Keep the old slot for immediate rollback. (5) Use **sticky settings** for slot-specific configuration that doesn't swap with the slot.

Q7. TE

What is the purpose of pipeline caching and how do you implement it?

Answer:

Pipeline caching reduces build time by storing and restoring dependencies between runs. Use the **Cache** task with a key (hash of dependency manifest files like package-lock.json). On a cache hit, the task restores the cache instead of downloading packages. Typical savings: 50-80% of dependency download time. Cache is stored per branch and key hash.

Q8. ADVANCED

How do you pass variables between pipeline stages?

Answer:

Use **output variables**: in the producing stage, set via **##vso[task.setvariable variable=name;isOutput=true]value**. In the consuming stage, reference as **stageDependencies.StageName.JobName.outputs['StepName.VariableName']**. For complex data, serialize to JSON or store in an artifact file passed between stages.

Q9. TE

What are Approvals and Checks in Azure Pipelines Environments?

Answer:

Controls on Environments: **Approvals** require designated users to manually approve a deployment. **Branch control** restricts deployments to specific branches. **Invoke Azure Function / REST API** checks call external systems for automated validation. **Business hours check** restricts deployments to specific time windows. All checks must pass before deployment proceeds.

Q10. TE

How do you implement parallel jobs in a YAML pipeline?

Answer:

Parallel jobs are configured using **strategy.matrix** to run the same job with different parameter combinations simultaneously (e.g., testing on multiple OS versions). Or define multiple jobs without dependsOn to run concurrently. The number of parallel jobs is limited by your Azure DevOps concurrency license (free tier: 1 parallel job).

Q11. TE

What is the purpose of a Release Gate in Classic pipelines?

Answer:

Release Gates are automated evaluations that continuously poll external sources (Azure Monitor alerts, work item queries, REST APIs) until the condition is met or timeout expires. Gates automate quality checks like verifying no active P1 incidents, all work items closed, or health check endpoints returning 200 before deployment proceeds.

Q12. TE

How do you configure pipeline artifacts to be used across stages?

Answer:

Publish artifacts using **PublishPipelineArtifact** with a named artifact. In subsequent stages, download using **DownloadPipelineArtifact** referencing the artifact name. Deployment jobs automatically download all artifacts from the current pipeline by default. Artifacts are immutable and versioned per pipeline run, ensuring consistency across stages.

Q13. ADVANCED

What is a YAML template extends pattern?

Answer:

The **extends** keyword enforces that a pipeline must extend a specified template, used for organizational security and compliance. The template defines required structure (security scans, compliance checks) that child pipelines cannot remove. Combined with required templates on protected branches, this ensures all production deployments pass mandatory security gates.

Q14. ADVANCED

How do you implement rollback in Azure Pipelines?

Answer:

Rollback approaches: (1) App Service deployment slots for instant slot-swap rollback. (2) For AKS, use **kubectl rollout undo** or maintain Helm release history. (3) Keep previous artifact versions in Azure Artifacts. (4) Design idempotent deployment scripts. (5) Trigger the previous successful pipeline run's deploy stage manually with environment approval override.

Q15.

BEGINNER

What is a self-hosted agent and when would you choose it over Microsoft-hosted?

Answer:

Choose self-hosted when you need: **access to private network resources** (internal databases, on-premises APIs), **custom software or tools** not on Microsoft-hosted images, **persistent caches** for faster builds, **GPU compute** for ML workflows, or **longer job timeouts** (Microsoft-hosted limit: 60 min free, 6 hours paid). Self-hosted agents also allow IP whitelisting.

Section 3: Infrastructure as Code (IaC)

Infrastructure as Code is a foundational DevOps practice enabling reproducible, version-controlled infrastructure provisioning. This section covers ARM templates, Bicep, Terraform, and their integration with Azure Pipelines for automated infrastructure delivery.

Q1. BEGINNER

What is Infrastructure as Code and what are its key benefits?

Answer:

IaC manages and provisions infrastructure through machine-readable configuration files. Key benefits: **Version control** (track changes, rollback), **Consistency** (identical environments every time), **Repeatability** (same config across Dev/Staging/Prod), **Automation** (CI/CD integration), and **Documentation** (code is the documentation of infrastructure state).

Q2. BEGINNER

What is the difference between ARM templates, Bicep, and Terraform for Azure?

Answer:

ARM templates are JSON-based Azure-native IaC — verbose but comprehensive coverage. **Bicep** is a DSL compiling to ARM JSON with cleaner syntax, type safety, and modules — Microsoft's recommended approach. **Terraform** is provider-agnostic HCL-based IaC supporting multi-cloud with a mature ecosystem, state management, and plan/apply workflow requiring a remote state backend for teams.

Q3. BEGINNER

What is Bicep and how does it improve over raw ARM templates?

Answer:

Bicep transpiles to ARM JSON. Improvements: **Clean syntax** (no JSON boilerplate), **Modules** for code reuse, **Type system** catching errors at compile time, **Auto-complete and linting** in VS Code, **Smaller files** (typically 40% the size of equivalent ARM JSON). Bicep is first-class in Azure CLI and Pipelines and is Microsoft's recommended IaC approach.

Q4. TE

How do you manage Terraform state in Azure?

Answer:

Store Terraform state in an **Azure Storage Account** using the azurerm backend. Configure storage_account_name, container_name, key, resource_group_name in backend.tf. Enable **blob soft delete and versioning** for state recovery. State locking is automatic via Azure Blob lease, preventing concurrent modifications. For sensitive state values, integrate with Azure Key Vault.

Q5. TE

What is the Terraform plan/apply workflow and why is it important?

Answer:

terraform plan generates an execution plan showing changes without applying them. **terraform apply** executes the plan. In CI/CD: CI stage runs plan (saved as binary plan file), plan output is reviewed, CD stage runs apply using the saved plan file. This separates intent (plan) from execution (apply) and prevents unintended changes from reaching production infrastructure.

Q6. TE

How do you implement IaC in Azure Pipelines?

Answer:

For Bicep: use **AzureCLI** or **AzureResourceManagerTemplateDeployment** task with Incremental or Complete mode. For Terraform: use **TerraformInstaller** and **TerraformTaskV4** tasks for init, plan, apply. Store backend config in pipeline variables. Use separate pipeline stages for plan (PR trigger) and apply (merge trigger). Gate production applies with environment approvals.

Q7. TE

What is the difference between Incremental and Complete deployment modes in ARM?

Answer:

Incremental mode (default) deploys only resources defined in the template, leaving existing resources not in the template untouched. **Complete mode** deploys all template resources and **deletes** any resources in the resource group not defined in the template. Complete mode ensures the template is the complete desired state but requires caution as it can delete unintended resources.

Q8. ADVANCED

What is the Terraform import command and when would you use it?

Answer:

terraform import brings existing infrastructure under Terraform management without destroying and recreating it. Use when: adopting IaC for manually-created resources, migrating from another IaC tool, or recovering from state drift. Run **terraform import resource_type.name resource_id**, then write matching HCL configuration to reflect the imported resource's attributes.

Q9. TE

How do you handle secrets in IaC?

Answer:

Never hardcode secrets in IaC files. Approaches: (1) **Azure Key Vault** references in ARM/Bicep (secrets resolved at deployment time). (2) **Terraform data sources** to read Key Vault secrets at plan/apply time. (3) **Pipeline secret variables** passed as parameters. (4) Use **Managed Identities** instead of credentials where possible. (5) Mark sensitive Terraform outputs with **sensitive = true**.

Q10. ADVANCED

What are Terraform Workspaces and how do they support multi-environment IaC?

Answer:

Terraform Workspaces create isolated state files within the same backend, enabling the same configuration to manage multiple environments. Each workspace (dev, staging, prod) has its own state. Use **terraform.workspace** in HCL to vary resource names or sizes per environment. Workspaces share the same codebase, so environment-specific differences are managed via variables or conditional expressions.

Q11. TE

What is idempotency in IaC and why does it matter?

Answer:

Idempotency means applying the same IaC configuration multiple times produces the same result — subsequent applies make no changes if infrastructure already matches the desired state. ARM, Bicep, and Terraform are inherently idempotent. This is critical for safe pipeline re-runs, drift correction, and disaster recovery, ensuring that re-applying IaC is always safe and predictable.

Q12. ADVANCED

What is policy-as-code and how is it implemented in Azure DevOps?

Answer:

Policy-as-code defines compliance rules in code evaluated automatically. In Azure: store **Azure Policy definitions** in Git as JSON/Bicep, deploy via Pipeline, enforce on subscriptions. Use **OPA (Open Policy Agent)** with Conftest to validate IaC files in PR pipelines before deployment. This prevents non-compliant infrastructure from being provisioned, shifting compliance left.

Q13. ADVANCED

How do you test IaC before deploying to production?

Answer:

Test IaC using: (1) **Linting** (bicep build, terraform validate, tflint). (2) **Unit tests** using Pester for ARM/Bicep or Terratest for Terraform. (3) **What-if analysis** (ARM what-if, terraform plan) to preview changes. (4) **Deploy to a test subscription** first and run integration tests. (5) **Policy compliance checks** using Azure Policy in audit mode before enforcement.

Q14. ADVANCED

What is Azure Deployment Stack?

Answer:

Azure Deployment Stacks manages a group of resources as a single atomic unit with lifecycle management. Unlike regular deployments, stacks track all deployed resources, can **delete unmanaged resources** (deny settings), and **prevent modifications** outside the stack. This bridges the gap between ARM incremental mode and Terraform's state management for full lifecycle control.

Q15.

TE

What is the Ansible role and how does it complement Terraform?

Answer:

Ansible is a configuration management tool using YAML playbooks. While Terraform provisions infrastructure (VM, network, storage), Ansible handles **configuration management** (install software, configure OS, deploy app configs) on those resources. Ansible is agentless (uses SSH/WinRM), idempotent, and integrates with Azure Pipelines via the AzureCLI task running ansible-playbook commands.

Section 4: Monitoring & Logging

Continuous monitoring closes the DevOps loop by providing feedback from production to development. This section covers Azure Monitor, Application Insights, Log Analytics, alerts, and dashboards for observing application health and pipeline performance.

Q1. BEGINNER

What is Azure Monitor and what does it collect?

Answer:

Azure Monitor is the unified observability platform for Azure and hybrid environments. It collects: **Metrics** (numerical time-series from resources, every minute by default), **Logs** (structured records in Log Analytics Workspace), **Traces** (distributed tracing via Application Insights), and **Activity Log** (subscription-level management events). Data is analyzed via Metrics Explorer, Log Analytics, and dashboards.

Q2. BEGINNER

What is Application Insights and what does it monitor?

Answer:

Application Insights is an APM service within Azure Monitor. It monitors: **Request rates and response times**, **Dependency calls** (SQL, HTTP), **Exceptions and stack traces**, **Custom events and metrics**, **User sessions and page views**, **Availability tests**, and **Live Metrics** for real-time monitoring. Instrumentation is via SDK, auto-instrumentation, or OpenTelemetry.

Q3. TE

What is KQL and how is it used in Azure DevOps contexts?

Answer:

KQL (Kusto Query Language) is the query language for Azure Monitor Log Analytics, Application Insights, and Microsoft Sentinel. In DevOps contexts: query build failure rates, deployment frequency, pipeline duration trends, and error spikes post-deployment. Key operators: **where** (filter), **summarize** (aggregate), **join** (combine tables), **project** (select columns), **render** (visualize). KQL is also used in pipeline release gates.

Q4. TE

How do you implement distributed tracing in a microservices application on Azure?

Answer:

Implement using: (1) **Application Insights SDK** or **OpenTelemetry** in each service. (2) Propagate **correlation IDs** (W3C TraceContext) across HTTP and message queue calls. (3) Use the **Application Map** in Application Insights to visualize service dependencies and failure rates. (4) Query the **requests, dependencies, and exceptions** tables in Log Analytics to trace end-to-end flows across services.

Q5. BEGINNER

What is the difference between Azure Monitor Metrics and Logs?

Answer:

Metrics are lightweight, numerical, time-series values stored in a time-series database with sub-minute granularity, ideal for real-time alerting with very low latency (93-day default retention). **Logs** are rich structured records in Log Analytics Workspace, queryable via KQL, ideal for troubleshooting and correlating events (configurable retention: 30 days to 7 years). Metrics alert faster; Logs provide richer context.

Q6. BEGINNER

How do you configure alerts in Azure Monitor?

Answer:

Azure Monitor Alerts consist of: **Alert Rule** (condition + target resource), **Condition** (metric threshold, log query result, activity log event), **Action Group** (what to do: email, SMS, push notification, webhook, Azure Function, Logic App, ITSM). Alert types: Metric alerts, Log alerts (KQL-based), Activity Log alerts, and Smart Detection (Application Insights anomaly detection).

Q7. ADVANCED

What are DORA metrics and how do you measure them in Azure?

Answer:

DORA (DevOps Research and Assessment) metrics: **Deployment Frequency** (query pipeline runs in Log Analytics), **Lead Time for Changes** (commit to production time via Azure DevOps Analytics), **Change Failure Rate** (% of deployments causing failures via Application Insights exception rates post-deploy), **Time to Restore Service** (MTTR from incident to recovery via Azure Monitor alert resolution times).

Q8. TE

How do you monitor Azure Pipelines performance?

Answer:

Monitor via: (1) **Azure DevOps Analytics** — pipeline pass rate, duration trends, agent utilization. (2) **Pipeline Insights** (built-in dashboard) — failure rates, slow tasks, flaky tests. (3) Export pipeline logs to **Azure Monitor** via Diagnostic Settings. (4) Use **Azure DevOps REST API** to extract metrics into custom dashboards. (5) Set up **Notifications** for pipeline failures to Teams or email.

Q9. ADVANCED

What is Log Analytics Workspace and how should it be architected?

Answer:

A Log Analytics Workspace is the central repository for Azure Monitor log data. Architecture considerations: **Centralized** (one workspace — easier cross-resource queries, single RBAC) vs **Distributed** (workspace per team/region — data sovereignty, cost allocation). Use **Resource Context access mode** for resource-level security. Plan retention (free 30 days, paid up to 7 years) and data ingestion costs.

Q10. TE

How do you implement availability monitoring for web applications?

Answer:

Use Application Insights **Availability Tests**: (1) **URL ping test** — simple HTTP check from multiple global locations. (2) **Standard test** — validates SSL cert, response time SLA, HTTP response codes. (3) **Custom TrackAvailability** — coded test with complex user flows. Configure alerts on availability below 50% or failures from multiple locations. Integrate with Azure Pipelines as post-deployment validation gates.

Q11. ADVANCED

How do you use Azure Monitor to detect deployment-related regressions?

Answer:

Detect post-deployment regressions by: (1) **Annotating deployments** in Application Insights (Release Annotations via Pipelines task). (2) Setting up **Smart Detection** for anomaly detection. (3) Creating **metric alerts** on error rate increases post-deployment. (4) Comparing metrics before/after deployment using time-range analysis in Metrics Explorer. (5) Using Application Insights **Deployment Markers** to correlate code changes with telemetry trends.

Q12. TE

What is Azure Monitor Workbooks and how are they used in DevOps?

Answer:

Azure Monitor Workbooks are interactive, parameterized reports combining text, KQL queries, metrics, and visualizations. In DevOps: build deployment frequency dashboards, DORA metrics reports, post-deployment health checks, and operational runbooks. Workbooks support drill-down navigation and can be shared across teams via Azure RBAC. Pre-built workbook templates exist for Container Insights, VM Insights, and security analysis.

Q13. ADVANCED

How do you centralize logging for containerized applications in AKS?

Answer:

Centralize AKS logs using: (1) Enable **Container Insights** (Azure Monitor agent as DaemonSet). (2) Collect **stdout/stderr** from all containers, node metrics, and Kubernetes events. (3) Configure **diagnostic settings** to send AKS control plane logs to Log Analytics. (4) Use **Azure Managed Grafana** for application-level metrics via Prometheus. (5) Implement structured logging (JSON) in applications for efficient KQL querying.

Q14. ADVANCED

What is Azure Monitor Private Link for secure log ingestion?

Answer:

Azure Monitor Private Link Scope (AMPLS) connects Log Analytics Workspaces and Application Insights to your VNet via Private Endpoints. All monitoring data (agent uploads, SDK telemetry) flows through Azure backbone without internet exposure. Critical for compliance in regulated environments. On-premises agents can send logs via ExpressRoute or VPN to the private endpoint, eliminating public internet ingestion.

Q15.

TE

What is the difference between proactive and reactive monitoring?

Answer:

Reactive monitoring responds to incidents after they occur — alert fires, team investigates, resolves. **Proactive monitoring** predicts and prevents incidents — capacity planning alerts, anomaly detection before thresholds breach, synthetic monitoring, chaos engineering with monitoring. Azure DevOps teams use both: reactive via Azure Monitor Alerts, proactive via Application Insights Smart Detection and capacity planning dashboards.

Section 5: Security & Compliance

Security must be integrated throughout the DevOps lifecycle — not bolted on at the end. This section covers DevSecOps practices, secret management, compliance automation, container security, and supply chain security for building secure delivery pipelines.

Q1. BEGINNER

What is DevSecOps and how is it implemented in Azure DevOps?

Answer:

DevSecOps integrates security throughout the DevOps pipeline. Implementation: (1) **SAST** (SonarQube, Checkmarx) in CI. (2) **DAST** (OWASP ZAP) post-deployment. (3) **Dependency scanning** (OWASP Dependency Check, Snyk). (4) **Container scanning** (Trivy, Defender for Containers). (5) **Secret detection** (git-secrets, Defender for DevOps). (6) **IaC security** (tfsec, Checkov) in PR pipelines.

Q2. BEGINNER

How do you manage secrets securely in Azure DevOps pipelines?

Answer:

Secure secret management: (1) Store secrets in **Azure Key Vault** linked to Pipeline Variable Groups. (2) Use **Managed Identities** for service connections — eliminates credential rotation. (3) Mark pipeline variables as **secret** (masked in all logs). (4) Never print secrets via echo. (5) Rotate secrets regularly using Key Vault versioning. (6) Restrict Variable Group access via pipeline RBAC. (7) Use Workload Identity Federation for service connections.

Q3. TE

What is Microsoft Defender for DevOps?

Answer:

Microsoft Defender for DevOps provides security posture management across DevOps platforms (Azure DevOps, GitHub, GitLab). It performs: **IaC scanning** (ARM, Bicep, Terraform misconfigurations), **Secret scanning** (leaked credentials in repos), **Dependency vulnerability scanning**, and **Container image scanning**. Findings surface in Defender for Cloud and can be added as PR annotations to block vulnerable code from merging.

Q4. BEGINNER

How do you implement branch protection policies in Azure Repos?

Answer:

Branch policies in Azure Repos: (1) **Require pull requests** (no direct pushes to main). (2) **Minimum reviewers** (e.g., 2 required approvals). (3) **Required builds** (CI must pass before merge). (4) **Comment resolution** (all PR comments must be resolved). (5) **Work item linking** (PRs must reference a work item). (6) **Merge strategy enforcement** (squash or rebase for clean history).

Q5. TE

What is SAST and DAST and how do you integrate them into pipelines?

Answer:

SAST (Static Application Security Testing) analyzes source code without execution. Integrate in CI via SonarQube, Checkmarx, or CodeQL tasks — runs on every build. **DAST (Dynamic Application Security Testing)** tests a running application by simulating attacks. Integrate post-deployment to a test environment using OWASP ZAP or Burp Suite. SAST catches issues early; DAST finds runtime vulnerabilities not visible in static analysis.

Q6. TE

How do you implement container image scanning in Azure Pipelines?

Answer:

Container scanning in pipelines: (1) Use **Trivy** to scan images for OS and library CVEs before pushing to registry. (2) Enable **Microsoft Defender for Containers** in ACR for continuous scanning. (3) Use **ACR Tasks** to automatically scan on push. (4) Fail the pipeline on critical/high severity findings using exit code checks. (5) Use **image signing** (Notary) to prevent unsigned images from being deployed.

Q7. ADVANCED

What is software supply chain security and how is it addressed in Azure?

Answer:

Software supply chain security protects integrity of software from source to deployment. Azure approaches: (1) **Azure Artifacts** with upstream policies blocking unverified packages. (2) **Defender for DevOps** for dependency scanning. (3) **SBOM (Software Bill of Materials)** generation using Microsoft SBOM Tool in pipelines. (4) **Code signing** for artifacts. (5) **SLSA framework** compliance via hermetic builds and provenance attestation.

Q8. ADVANCED

How do you implement compliance-as-code in Azure DevOps?

Answer:

Compliance-as-code: (1) Store **Azure Policy definitions** in Git and deploy via Pipelines. (2) Use **Conftest + OPA** to validate IaC against compliance rules in PRs. (3) Run **Defender for Cloud compliance assessments** as pipeline gates. (4) Generate **compliance reports** automatically from Azure Policy compliance data. (5) Use **Microsoft Purview** for data compliance classification. (6) Store evidence in Azure Storage for audit purposes.

Q9. TE

What is Azure AD Managed Identity and how does it improve pipeline security?

Answer:

Managed Identities provide Azure services with auto-managed identities in Azure AD, eliminating stored credentials. In pipelines: self-hosted agents on Azure VMs or VMSS use **System-assigned managed identity** to authenticate to Azure services (Key Vault, Storage, ACR) without credentials in pipeline YAML. This eliminates credential rotation, secret leakage risks, and manual service principal management.

Q10. ADVANCED

How do you audit and monitor security events in Azure DevOps?

Answer:

Audit via: (1) **Azure DevOps Audit Logs** — tracks permission changes, pipeline approvals, service connection modifications. Export to Azure Monitor or Sentinel. (2) **Azure AD Sign-in Logs** — tracks authentication to Azure DevOps. (3) **Pipeline run logs** with retention policies. (4) **Git history** for code changes. (5) Integrate with **Microsoft Sentinel** for SIEM correlation and automated incident response playbooks.

Q11. ADVANCED

What is secret rotation and how do you automate it?

Answer:

Secret rotation replaces credentials on a schedule to limit exposure windows. Automate in Azure: (1) Enable **Key Vault automatic rotation** for storage account keys, certificates. (2) Use **Key Vault event notifications** via Event Grid to trigger Logic Apps or Azure Functions when a secret nears expiry. (3) In pipelines, always reference secrets by Key Vault name — rotated secrets are automatically used on the next pipeline run without YAML changes.

Q12. ADVANCED

What is Workload Identity Federation for Azure DevOps service connections?

Answer:

Workload Identity Federation enables Azure DevOps service connections to authenticate to Azure using **federated credentials** — no client secret or certificate required. Azure DevOps exchanges an OIDC token (issued per pipeline run) for an Azure AD access token. Benefits: **no credential rotation**, **no secret storage**, **short-lived tokens per job**, and improved security over traditional service principal secrets. Now the default for new ARM service connections.

Q13. TE

What is the principle of least privilege in Azure DevOps?

Answer:

Least privilege in Azure DevOps: (1) Service connections use **service principals or managed identities** with only the permissions needed (e.g., Contributor on a specific resource group). (2) Pipeline agents run as low-privilege accounts. (3) Variable groups have **pipeline-level permissions** restricting which pipelines can use them. (4) Use **project-scoped service connections** rather than organization-scoped for blast radius reduction.

Q14. TE

How do you prevent sensitive data from being logged in pipelines?

Answer:

Prevent log exposure: (1) Declare secrets as **pipeline secret variables** — Azure DevOps automatically masks them in logs. (2) Use **##vso[task.setvariable variable=name;issecret=true]** for dynamically computed secrets. (3) Enable **agent diagnostic logs** only in controlled environments. (4) Set appropriate log retention policies. (5) Review pipeline YAML for accidental echo of environment variables containing sensitive values.

Q15.

ADVANCED

What is container runtime security and how do you implement it for AKS?

Answer:

Container runtime security in AKS: (1) **Microsoft Defender for Containers** — detects anomalous container behavior, privilege escalation, crypto mining. (2) **Kubernetes admission controllers** — Azure Policy Add-on enforces pod security standards (no privileged containers, read-only root filesystem). (3) **Network policies** — restrict pod-to-pod communication. (4) **Workload identity** — pods authenticate via managed identity without stored credentials.

Section 6: Git & Version Control

Git is the foundation of modern DevOps. This section covers branching strategies, pull request workflows, repository management in Azure Repos, and best practices for maintaining a clean, collaborative codebase aligned with continuous delivery.

Q1. BEGINNER

What is the difference between Git merge, rebase, and squash?

Answer:

Merge combines branches by creating a merge commit, preserving full history of both branches. **Rebase** rewrites commits from a branch onto the tip of another branch, creating linear history (don't rebase shared branches). **Squash** combines all commits from a feature branch into a single commit before merging, keeping the main branch history clean and atomic per feature.

Q2. TE

What are the main Git branching strategies and when to use each?

Answer:

GitFlow: main, develop, feature, release, hotfix branches — suited for scheduled release cycles. **GitHub Flow**: main + short-lived feature branches merged via PR — simple, suited for continuous delivery. **Trunk-Based Development**: all developers commit to trunk (main) with feature flags for incomplete features — best for high-frequency continuous deployment. AZ-400 aligns Trunk-Based Development with CI/CD best practices.

Q3. BEGINNER

What is a Pull Request and what should a PR workflow include?

Answer:

A PR proposes, reviews, and integrates code changes. A robust PR workflow includes: **Automated CI build** (must pass before merge), **Required reviewers** (minimum 2), **Code coverage check**, **Linting/formatting check**, **Security scan**, **Work item link**, **Comment resolution** requirement, and **Merge strategy enforcement** (squash or rebase). Configure all via Azure Repos Branch Policies.

Q4. TE

What is Git cherry-pick and when would you use it?

Answer:

git cherry-pick applies changes from a specific commit to the current branch without merging the entire source branch. Use cases: applying a critical hotfix to a release branch, backporting a specific bug fix to an older version, or selectively applying a feature commit. Caution: cherry-picking duplicates commits (different SHA), which can cause conflicts when branches are later merged.

Q5. TE

What is semantic versioning and how do you implement it in pipelines?

Answer:

Semantic versioning (SemVer) uses **MAJOR.MINOR.PATCH**: MAJOR = breaking changes, MINOR = new features (backward-compatible), PATCH = bug fixes. Implement in pipelines: (1) Use **GitVersion** tool to auto-calculate version based on commit history and branch conventions. (2) Apply version to NuGet packages, Docker images, and release notes automatically in CI. (3) Use Conventional Commits to drive automatic version bumping.

Q6. ADVANCED

How do you implement a monorepo strategy with Azure DevOps?

Answer:

Monorepo in Azure DevOps: (1) Use **path-based triggers** in YAML to build only changed components. (2) Use **sparse checkout** to clone only relevant directories. (3) Organize code with **CODEOWNERS** files for PR routing. (4) Use **pipeline templates** shared across services. (5) Implement **affected service detection** using git diff to avoid unnecessary builds. Tools: Nx, Turborepo, Bazel for dependency graph analysis.

Q7. TE

What are Git hooks and how can they be used in DevOps?

Answer:

Git hooks are scripts that run at specific Git lifecycle events. Common uses: **pre-commit** (run linter, secret scanner, format code), **commit-msg** (enforce commit message format — Conventional Commits), **pre-push** (run unit tests before pushing). In Azure DevOps, use **Husky** for client-side hooks and **branch policies** with required builds for server-side enforcement.

Q8. TE

What is Conventional Commits and how does it enable automation?

Answer:

Conventional Commits specifies commit messages as **type(scope): description** where type is feat, fix, docs, chore, refactor, test, ci. Benefits: (1) **Automated changelog generation** using tools like Release Please. (2) **Automatic semantic versioning** — feat = MINOR bump, fix = PATCH bump, BREAKING CHANGE = MAJOR bump. (3) Better readability and searchability of Git history for all team members.

Q9. TE

What is Git LFS and when should you use it?

Answer:

Git LFS replaces large binary files in Git with lightweight text pointers, while storing actual file content in a separate LFS server. Azure Repos supports Git LFS natively. Use Git LFS for: **binary assets** (images, videos, ML models), **compiled binaries**, **design files** (Figma, PSD). Without LFS, large binaries bloat repository size, slow clone times, and degrade Git performance for all team members.

Q10. TE

How do you enforce commit message standards in Azure DevOps?

Answer:

Enforce commit standards via: (1) **Git commit-msg hook** with Commitlint for local enforcement. (2) **PR title validation** in Azure Repos branch policy requiring Conventional Commits format via a pipeline status check. (3) **Squash merge** enforced via branch policy — PR title becomes the single commit message. (4) **Pipeline linting step** validating commit messages for all commits in a PR.

Q11. ADVANCED

What is sparse checkout in Git and when is it useful in CI/CD?

Answer:

Sparse checkout allows cloning only a subset of a repository's directories. Useful in CI/CD when: working with a **monorepo** (checkout only the service being built, saving time and disk), **large repositories** with many unrelated components, or **resource-constrained build agents**. Configure in Azure Pipelines using the checkout task with **sparseCheckoutDirectories** option.

Q12. TE

How do you handle merge conflicts in a CI/CD environment?

Answer:

Handle merge conflicts by: (1) Enforcing **short-lived feature branches** (merged within 1-2 days). (2) Using **feature flags** to merge incomplete code safely. (3) Running **trunk-based development** to minimize long-running branches. (4) Using **automated conflict detection** in PR checks. (5) Rebasing feature branches on main daily to stay current and reduce divergence.

Q13. BEGINNER

What is the difference between git fetch and git pull?

Answer:

git fetch downloads changes from remote to local remote-tracking branches (origin/main) without modifying your working tree or local branches. **git pull** performs a fetch followed by a merge (or rebase with --rebase) into your current local branch. Prefer git fetch + manual review before git merge for better control, especially in team environments with potential conflicts.

Q14. ADVANCED

How do you manage Git repository size and performance in Azure Repos?

Answer:

Manage repo size by: (1) Using **Git LFS** for binary files. (2) Using **shallow clones** (fetchDepth: 1) in pipelines. (3) Avoiding committing compiled outputs, node_modules, or IDE files (.gitignore). (4) Running **git filter-repo** to remove accidentally committed large files from history. (5) Monitoring repo size in Azure DevOps project settings and setting alerts for size thresholds.

Q15.

ADVANCED

What is gitops and how does Git serve as the source of truth?

Answer:

GitOps is a DevOps pattern where Git is the **single source of truth** for both application code and infrastructure configuration. All changes to infrastructure and deployments are made via Git commits and PRs, not manual commands. An operator (Flux, ArgoCD) continuously reconciles the actual state to match the Git repository. Benefits: full audit trail in Git, rollback by reverting commits, and peer review of all infrastructure changes.

Section 7: Kubernetes & Containers

Containerization and Kubernetes orchestration are central to modern DevOps on Azure. This section covers Docker, AKS, Helm, GitOps, and container CI/CD pipelines — all heavily tested in the AZ-400 exam.

Q1. BEGINNER

What is the difference between a Docker image and a Docker container?

Answer:

A **Docker image** is an immutable, read-only template containing application code, runtime, libraries, and configuration — created by a Dockerfile. A **Docker container** is a running instance of an image with a writable layer on top. Multiple containers can run from the same image. Images are pushed to registries (ACR); containers run on hosts or orchestrators (AKS).

Q2. BEGINNER

What is Azure Kubernetes Service (AKS) and what does it manage?

Answer:

AKS is a fully managed Kubernetes service. Azure manages: **control plane** (kube-apiserver, etcd, scheduler, controller-manager) — provided at no cost. You manage: **worker nodes** (node pools — VM SKUs, OS, autoscaling). AKS integrates with Azure AD (RBAC), Azure Monitor (Container Insights), Azure CNI (networking), Key Vault (secrets), and ACR (container images).

Q3. BEGINNER

What is Helm and what problem does it solve?

Answer:

Helm is the package manager for Kubernetes. It uses **Charts** (collections of YAML templates) to deploy and manage complex applications as a single unit. Helm solves: **templating** (parameterize manifests for different environments), **versioning** (rollback to previous chart versions), **dependency management** (charts depending on other charts), and **lifecycle management** (install, upgrade, rollback, uninstall via Helm releases).

Q4. BEGINNER

How do you build a Docker image in Azure Pipelines and push it to ACR?

Answer:

In YAML pipeline: (1) Use **Docker@2 task** with command: buildAndPush, repository: myacr.azurecr.io/myapp, tags: $(Build.BuildId). (2) Set containerRegistry to an ACR service connection. (3) Best practices: use **multi-stage Dockerfile** to minimize image size, pin base image digests for reproducibility, and tag images with both the build ID and git SHA for full traceability across environments.

Q5. TE

What is a multi-stage Dockerfile and why is it recommended?

Answer:

A multi-stage Dockerfile uses multiple FROM statements to separate build and runtime environments. Stage 1 (builder) uses sdk image, runs compilation. Stage 2 (runtime) uses smaller runtime image, copies only published output. Benefits: **smaller final images** (runtime image excludes build tools), **better security** (fewer packages = smaller attack surface), **faster deployments**, and no build tools exposed in production images.

Q6. TE

How do you deploy to AKS using Azure Pipelines?

Answer:

Deploy using: (1) **KubernetesManifest task** with action deploy, manifests pointing to Kubernetes YAML files, imagePullSecrets for ACR access. (2) Or **HelmDeploy task** for Helm-based deployments. (3) Use an **Environment** resource pointing to an AKS namespace — records deployment history and enables approvals. (4) Use **image substitution** to replace the image tag in manifests with the freshly built image SHA.

Q7. ADVANCED

What is GitOps and how is it implemented with AKS?

Answer:

GitOps uses Git as the single source of truth for infrastructure and application configuration. Changes are made via PRs; an operator continuously reconciles the cluster state to match Git. Azure implementation: (1) **AKS GitOps extension** (Flux v2 built into AKS). (2) Define Kubernetes manifests or Helm releases in Git. (3) Flux polls Git and applies changes automatically. (4) Drift detection alerts when cluster diverges from Git repository.

Q8. ADVANCED

What is AKS KEDA and how does it enable event-driven autoscaling?

Answer:

KEDA (Kubernetes Event-Driven Autoscaling) scales Kubernetes deployments based on external event sources beyond CPU/memory. Native KEDA is built into AKS. Scalers include: **Azure Service Bus queue depth**, **Azure Event Hub lag**, **Azure Storage Queue length**, **Prometheus metrics**, HTTP request count. Scales from 0 to N and back to 0, enabling cost-efficient event-driven microservices that consume no resources when idle.

Q9. TE

What is the AKS Cluster Autoscaler?

Answer:

The AKS Cluster Autoscaler automatically adjusts node count in a node pool based on pending pod demand. When pods can't be scheduled due to insufficient resources, it adds nodes. When nodes are underutilized and pods can be moved, it removes nodes. Configure with **min-count** and **max-count** per node pool. Works in conjunction with HPA (Horizontal Pod Autoscaler) for full bi-directional auto-scaling.

Q10. ADVANCED

How do you implement canary deployments in AKS?

Answer:

Canary deployments: (1) Deploy new version as a separate Deployment with small replica count (e.g., 10% of total replicas). (2) Use **Service with label selector** to split traffic proportionally based on replica counts. (3) For precise traffic splitting, use **Istio** (VirtualService weight) or **NGINX Ingress annotations**. (4) Monitor error rates and latency in Application Insights. (5) Gradually increase canary replicas if metrics healthy; rollback if not.

Q11. TE

What is Azure Container Registry (ACR) and what security features does it provide?

Answer:

ACR is Azure's managed container registry. Security features: (1) **Private endpoints** — restrict registry access to VNet. (2) **Managed Identity authentication** — no stored credentials. (3) **Defender for Containers** — vulnerability scanning on push and running images. (4) **Content trust** — sign images with Notary. (5) **ACR Tasks** — automated builds and scanning on base image update. (6) **Geo-replication** — pull from nearby registry replica.

Q12. BEGINNER

What is a Kubernetes ConfigMap vs Secret and how do you manage them securely in AKS?

Answer:

ConfigMap stores non-sensitive configuration data as key-value pairs, mounted as environment variables or volume files. **Secret** stores sensitive data base64-encoded (not encrypted in etcd by default). In AKS, use **Azure Key Vault Provider for Secrets Store CSI Driver** to mount Key Vault secrets directly into pods, avoiding storing sensitive data in etcd or Git repositories.

Q13. TE

How do you implement zero-downtime deployments in Kubernetes?

Answer:

Zero-downtime using: (1) **RollingUpdate strategy** with configured maxSurge and maxUnavailable. (2) **Readiness probes** — Kubernetes only routes traffic to pods that pass readiness checks. (3) **Liveness probes** — restarts unhealthy pods. (4) **PodDisruptionBudget** — ensures minimum available pods during voluntary disruptions. (5) **preStop hook** — graceful shutdown period before pod termination.

Q14. ADVANCED

What is AKS workload identity and why is it preferred?

Answer:

AKS Workload Identity uses **Federated Identity Credentials** allowing pods to authenticate to Azure services via Kubernetes service account token projection. Pods authenticate to Key Vault, Storage, ACR without stored credentials. Benefits: no credential management, no DaemonSet agent required, compatible with MSAL and Azure SDK auto-detection, and follows standard OIDC federation — replacing the deprecated Pod Identity approach.

Q15.

ADVANCED

How do you manage Kubernetes manifests across multiple environments in Azure DevOps?

Answer:

Manage manifests across environments by: (1) Using **Kustomize overlays** — base manifests + environment-specific patches (replica counts, resource limits, image tags). (2) Using **Helm values files** per environment. (3) Using **Azure DevOps variable substitution** in manifest files during pipeline execution. (4) Implementing GitOps with separate overlay directories per environment. (5) Storing environment-specific config in **Azure App Configuration** referenced at runtime.

Section 8: Azure Services Integration

AZ-400 engineers must understand how Azure DevOps integrates with the broader Azure ecosystem. This section covers Azure Artifacts, App Service, Function Apps, Service Bus, API Management, and other Azure services commonly used in DevOps workflows.

Q1. BEGINNER

What is Azure Artifacts and what package types does it support?

Answer:

Azure Artifacts is a package management service hosting private and public package feeds. Supported formats: **NuGet** (C#/.NET), **npm** (Node.js), **Maven** (Java), **PyPI** (Python), **Gradle** (Android/Java), **Universal Packages** (any file type). Artifacts supports **upstream sources** — proxying public registries so packages are cached and available even if upstream is down.

Q2. TE

What are Azure Artifacts upstream sources and why are they important?

Answer:

Upstream sources allow an Artifacts feed to proxy and cache packages from public registries. Importance: (1) **Resilience** — packages available even if public registry is down. (2) **Security** — only explicitly used packages are cached; unreviewed packages can be blocked. (3) **Audit trail** — all consumed packages are logged. (4) **Performance** — cached packages download faster from Azure backbone vs. public internet in CI agents.

Q3. TE

How do you configure Azure App Service deployment slots in a CI/CD pipeline?

Answer:

Deployment slot CI/CD: (1) **Deploy to staging slot** using AzureWebApp task with slotName: staging. (2) Run **smoke tests** against the staging slot URL. (3) Use **AzureAppServiceManage task** with action SwapSlots to swap staging □ production. (4) Production receives new code with zero downtime. (5) Keep staging slot as rollback — re-swap if production issues arise. Configure slot-specific settings as sticky (not swapped).

Q4. TE

What is Azure App Configuration and how does it integrate with DevOps?

Answer:

Azure App Configuration centralizes application settings and feature flags. DevOps integration: (1) Store environment-specific configs in App Configuration, referenced by apps at runtime. (2) Use **feature flags** for gradual feature rollouts (percentage filter, targeted release). (3) Reference Key Vault secrets in App Configuration. (4) Export configuration via pipeline for non-cloud apps. No redeployment needed for configuration changes.

Q5. TE

How do you automate Azure Function deployment using Azure Pipelines?

Answer:

Deploy Azure Functions using: (1) **AzureFunctionApp task** specifying function app name, package path (zip artifact from build), and runtime stack. (2) For containerized functions: build Docker image □ push to ACR □ AzureFunctionAppContainer task. (3) Use **deployment slots** for blue-green deployments. (4) Use **zip deployment** for serverless functions — faster than msdeploy for Functions.

Q6. TE

What is Azure API Management and how is it integrated into DevOps pipelines?

Answer:

APIM manages API gateway policies. DevOps integration: (1) Store **APIM policies in Git** as XML files. (2) Import API definitions (OpenAPI/Swagger) via **AzureCLI task** or **APIM REST API** in pipelines. (3) Use **Bicep/Terraform** to provision APIM infrastructure. (4) Test APIs in pipeline using Postman or Newman after deployment. (5) Deploy **Developer Portal** content via pipeline for self-service API documentation.

Q7. TE

How do you implement feature flags in a CI/CD workflow?

Answer:

Feature flags enable deploying incomplete features safely: (1) Use **Azure App Configuration feature flags** with filters (percentage rollout, user targeting). (2) Deploy code with feature flag off — merge to main immediately (trunk-based development). (3) Gradually enable flag via App Configuration without redeployment. (4) Monitor feature-specific metrics in Application Insights using custom dimensions. (5) Kill switch — disable flag immediately if issues detected without triggering a full rollback.

Q8. ADVANCED

How do you implement database deployments in Azure Pipelines?

Answer:

Database deployment strategies: (1) **DACPAC deployment** (SQL Server) via SqlAzureDacpacDeployment task — schema comparison and drift application. (2) **Migration scripts** (EF Core migrations, Flyway, Liquibase) — ordered SQL scripts applied incrementally. (3) Use a **separate DB stage** with approval before schema changes. (4) Run **backward-compatible migrations first**, deploy code, then cleanup migrations. (5) Always backup before applying schema changes in production.

Q9. TE

What is Azure Load Testing integration with Azure Pipelines?

Answer:

Azure Load Testing runs JMeter-based load tests as a pipeline stage. Integration: (1) Add AzureLoadTest task pointing to JMX test file and Azure Load Testing resource. (2) Define **failure criteria** (p95 response time, error rate thresholds). (3) Pipeline fails if criteria are not met. (4) Results appear in pipeline artifacts and Azure Load Testing portal. (5) Run as a **pre-production gate** to prevent performance regressions from reaching production.

Q10. BEGINNER

How do you use Azure CLI and Azure PowerShell in pipelines?

Answer:

Use Azure CLI via **AzureCLI task** (authenticates using service connection automatically). Use Azure PowerShell via **AzurePowerShell task**. Best practices: (1) Pin CLI/PowerShell version for reproducibility. (2) Use **--query** in CLI to extract specific values. (3) Store outputs in pipeline variables using **##vso[task.setvariable]**. (4) Use **--only-show-errors** to reduce log noise. (5) Test scripts locally before embedding in pipeline.

Q11. ADVANCED

What is Azure Event Hub and how do DevOps teams use it?

Answer:

Azure Event Hub is a high-throughput event streaming platform (millions of events/second). DevOps uses: (1) Stream **pipeline telemetry** to Event Hub for custom analytics. (2) Stream **application logs** via Diagnostic Settings for real-time processing. (3) Stream **Azure Monitor alerts** for automated incident response. (4) Use Event Hub as a **CI/CD pipeline trigger** — KEDA scales AKS workloads based on Event Hub consumer lag.

Q12. TE

How do you manage connection strings and app settings across environments in Azure DevOps?

Answer:

Manage environment-specific configuration by: (1) **Azure App Configuration** — environment-labeled settings referenced at runtime. (2) **Key Vault references** in App Service settings — settings resolve to Key Vault secrets automatically. (3) **Pipeline variable groups** per environment. (4) **Bicep parameters files** per environment for IaC settings. (5) Avoid storing connection strings in source code or pipeline YAML.

Q13. BEGINNER

What is Azure Static Web Apps and how is it deployed via DevOps?

Answer:

Azure Static Web Apps automatically builds and deploys full-stack web apps from Azure DevOps. The service auto-generates a **YAML pipeline** using AzureStaticWebApp task. It provides **staging environments per PR** (preview URLs), **globally distributed hosting**, managed Functions API backend, and built-in authentication. The pipeline handles build, test, and deployment in a single automated workflow.

Q14. TE

How do you integrate Azure Service Bus in pipeline testing?

Answer:

Service Bus pipeline testing: (1) Use a **dedicated dev Service Bus namespace** for integration tests. (2) Clean up test messages in pipeline teardown steps. (3) Use **dead-letter queue monitoring** as a post-deployment health check. (4) Validate message schema in pipeline using JSON Schema validation tasks before sending to Service Bus. (5) Use **Service Bus Explorer** in pipeline to inspect messages during test runs for debugging.

Q15. BEGINNER

What is Azure DevOps integration with Microsoft Teams?

Answer:

Azure DevOps integrates with Teams via: (1) **Azure DevOps app for Teams** — receive pipeline notifications, approve releases, view work items in Teams channels. (2) **Incoming webhook connectors** — custom pipeline notifications. (3) **Azure Boards integration** — link Teams tabs to Boards, create work items from Teams messages. (4) **Approval notifications** — environment approval requests appear in Teams for instant response without navigating to Azure DevOps portal.

Section 9: Real-World Scenario-Based Questions

Scenario-based questions test your ability to apply Azure DevOps knowledge to realistic engineering challenges. These questions require combining multiple concepts and making architectural trade-offs — critical for senior-level interviews and the AZ-400 exam.

Q1. ADVANCED

Your production deployment failed halfway through. How do you handle this situation?

Answer:

Response plan: (1) **Assess impact** — check Application Insights for error spike, Azure Monitor for alerts. (2) **Rollback immediately** — for App Service use slot swap back; for AKS run helm rollback or kubectl rollout undo; assess if DB schema changes are backward-compatible before rolling back code. (3) **Post-incident** — review pipeline logs, identify root cause. (4) **Prevention** — add smoke test stage before routing production traffic, implement canary deployment process.

Q2. ADVANCED

How would you design a CI/CD pipeline for a microservices application with 20 services?

Answer:

Design: (1) **Monorepo with path-based triggers** — each service has its own pipeline triggered only on changes to its directory. (2) **Shared pipeline templates** in a central repo for build, test, and deploy stages. (3) **Independent deployments** — each service deploys independently to its AKS namespace. (4) **Service mesh** (Istio) for traffic management. (5) **Geo-replicated ACR** for fast image pulls. (6) **Flux/ArgoCD** for GitOps-based deployment to AKS.

Q3. ADVANCED

A developer accidentally committed API keys to Git. What are your immediate steps?

Answer:

Immediate response: (1) **Rotate the exposed credentials immediately** — assume compromised. (2) **Revoke the old key** in the target service. (3) Remove the commit from Git history using **git filter-repo** or BFG Repo Cleaner — force-push to all branches. (4) **Check audit logs** for unauthorized use of the exposed key. (5) **Prevention** — enable Defender for DevOps secret scanning, implement pre-commit hooks with git-secrets, require Key Vault for all secrets.

Q4. ADVANCED

How would you implement a zero-downtime database schema migration in a CI/CD pipeline?

Answer:

Expand-Contract migrations: Phase 1 — add new column (nullable), deploy code that writes to both old and new columns. Phase 2 — migrate existing data. Phase 3 — deploy code that reads from new column only. Phase 4 — drop old column. Use **Flyway or Liquibase** for ordered migrations with checksums. Run migrations in a **separate pipeline stage** before code deployment with approval gate. Always test rollback procedures.

Q5. ADVANCED

Your team wants to reduce deployment frequency from monthly to daily. What do you recommend?

Answer:

Transformation roadmap: (1) **Trunk-Based Development** — eliminate long-lived branches, merge to main daily. (2) **Feature flags** — deploy incomplete features safely. (3) **Automated testing** — comprehensive unit, integration, and e2e tests. (4) **Blue-green deployments** — eliminate deployment downtime. (5) **Progressive delivery** — canary to 5% before full rollout. (6) **Automated rollback** on error threshold breach. (7) **DORA metrics dashboard** to track improvement.

Q6. ADVANCED

How would you set up a multi-environment deployment pipeline for a regulated industry (e.g., banking)?

Answer:

Regulated pipeline: (1) **Environments with mandatory approvals** from compliance officers for production. (2) **ITSM integration** (ServiceNow) — change request auto-created and required for prod. (3) **Immutable artifacts** — same image SHA deployed through all environments. (4) **Full audit trail** — all deployments logged with approver, time, and changeset. (5) **Automated compliance scans** as mandatory gates. (6) **Segregation of duties** — different teams for build vs. deploy to production.

Q7. ADVANCED

Your build times have grown from 5 minutes to 45 minutes. How do you investigate and fix this?

Answer:

Investigation: (1) **Pipeline analytics** — identify which task is slowest using Insights tab. (2) **Common culprits** — dependency restoration (fix with caching), test suite growth (fix with parallel testing), Docker builds (fix with layer caching and BuildKit). (3) **Solutions** — implement pipeline caching for dependencies, run tests in parallel using matrix strategy, use self-hosted agents with pre-pulled Docker images, enable Docker BuildKit with cache-from, move slow tests to a separate nightly pipeline.

Q8. ADVANCED

How would you implement disaster recovery for your Azure DevOps organization itself?

Answer:

Azure DevOps DR: (1) **Regular backups** — export Azure Repos via git clone --mirror, export work items via REST API, YAML pipelines are already in Git. (2) **Pipeline templates in separate repo** — independent backup. (3) **Self-hosted agents on VMSS** — recreatable via IaC. (4) **Azure AD as identity provider** — survives Azure DevOps outage. (5) Consider **GitHub as secondary VCS mirror** for critical repos. (6) Document recovery runbooks in Azure DevOps Wiki (export periodically).

Q9. TE

A pipeline is passing locally but failing in CI. How do you debug this?

Answer:

Debug approach: (1) **Compare environments** — check agent OS, SDK version, tool versions. Pin tool versions in pipeline. (2) **Enable verbose logging** — set system.debug=true in pipeline variables. (3) **Reproduce on fresh machine** — test in Docker container matching agent image. (4) **Check environment variables** — CI may lack env vars present locally. (5) **Examine artifact paths** — path separators differ between OS. (6) Run pipeline with **self-hosted agent on local machine** to narrow environment differences.

Q10. ADVANCED

How do you handle different teams needing different levels of access to the same pipeline?

Answer:

Access control design: (1) **Security groups** in Azure DevOps (Project Contributors, Release Approvers, Readers). (2) **Environment permissions** — only designated approvers can approve prod environment deployments. (3) **Variable group RBAC** — restrict access to production secret variable groups. (4) **Service connection scope** — prod service connection restricted to prod pipeline only. (5) **Protected resources** — environment, service connection, variable group requiring explicit authorization per pipeline.

Q11. TE

Your microservice's Docker image is 2GB. How do you optimize it?

Answer:

Image optimization: (1) **Multi-stage builds** — final stage uses minimal base image (alpine, distroless). (2) **Choose minimal base images** — alpine (5MB) vs ubuntu (72MB), distroless for production. (3) **Minimize layers** — combine RUN commands with &&. (4) **Use .dockerignore** — exclude node_modules, .git, test files. (5) **Remove build dependencies** — don't install gcc/make in final stage. (6) **Analyze with dive tool** to identify large layers.

Q12. ADVANCED

How would you implement a global deployment pipeline that deploys to 10 Azure regions?

Answer:

Global deployment: (1) **Geo-replicated ACR** — image pulled from nearest replica. (2) **Matrix strategy** — parallel deployment to all regions simultaneously or sequential with health check gates. (3) **Azure Front Door** — global load balancing post-deployment. (4) **Rolling regional deployment** — deploy to one region, validate, proceed. (5) **Cosmos DB multi-region writes** — data layer supports global deployment. (6) **Region-specific config** via App Configuration labels.

Q13. ADVANCED

How do you measure and improve the quality of your CI/CD pipeline?

Answer:

Pipeline quality metrics: (1) **DORA metrics** — deployment frequency, lead time, MTTR, change failure rate. (2) **Pipeline reliability** — percentage of builds passing, flaky test rate. (3) **Build duration trends** — tracked in pipeline analytics. (4) **Security gate pass rate** — how often security scans block builds. (5) Improve by: investing in test quality (eliminate flaky tests), caching, parallelization, and feedback loop shortening via pre-commit hooks.

Q14. ADVANCED

A team is deploying the same artifact differently to each environment. Is this a problem?

Answer:

Yes — this is an anti-pattern. The principle of **Build Once, Deploy Everywhere** means the same artifact (same Docker image SHA, same NuGet package version) should be deployed identically across all environments. Environmental differences should come from **external configuration** (App Configuration, Key Vault, environment variables) — not from rebuilding the artifact per environment. Rebuilding creates risk of environment-specific bugs reaching production undetected.

Q15. ADVANCED

How do you onboard a new team to Azure DevOps quickly and securely?

Answer:

Onboarding checklist: (1) **Project template** — pre-configured project with process template, area paths, iteration paths. (2) **Pipeline templates** — starting point YAML templates for build/deploy. (3) **Security groups** — auto-assign new members to appropriate groups. (4) **Variable group** with non-sensitive shared values. (5) **Branch policies** — applied via Azure DevOps ARM API in onboarding pipeline. (6) **Wiki documentation** — runbook for first deployment and contribution guidelines.

Section 10: Troubleshooting & Debugging

Troubleshooting is a critical real-world skill for AZ-400 engineers. This section covers diagnosing pipeline failures, agent issues, deployment problems, and performance bottlenecks — testing the depth of practical Azure DevOps experience.

Q1. BEGINNER

How do you enable detailed logging in Azure Pipelines for debugging?

Answer:

Enable detailed logging: (1) Set pipeline variable **system.debug=true** — enables verbose output from all tasks. (2) Set **agent.diagnostic=true** — logs agent infrastructure details. (3) Download **diagnostic logs** from pipeline run summary page (includes agent, worker, and task logs). (4) For self-hosted agents, check agent _diag folder logs. (5) Re-run failed job with **Re-run failed jobs** to reproduce on same agent if state-dependent.

Q2. TE

A self-hosted agent shows as offline in Azure DevOps. What are your troubleshooting steps?

Answer:

Troubleshoot offline agent: (1) Check agent service is running: **services.msc** (Windows) or **systemctl status vsts.agent** (Linux). (2) Check agent logs in _diag folder for authentication or connectivity errors. (3) Verify **Personal Access Token** is valid — re-configure agent if expired. (4) Check network connectivity to **dev.azure.com** and **vstsagentpackage.azureedge.net**. (5) Verify firewall rules allow outbound 443. (6) Re-register agent if all else fails.

Q3. TE

A pipeline task fails with 'Authorization failed'. How do you diagnose this?

Answer:

Authorization failure diagnosis: (1) Identify which **service connection** the task uses. (2) Check service connection is **authorized for this pipeline** (Project Settings □ Service Connections □ Edit □ Security). (3) Verify the service principal has required Azure RBAC role on the target resource. (4) Check if the service principal's **client secret has expired** (App Registrations □ Certificates & Secrets). (5) For managed identity connections, verify the agent VM's identity has the required role.

Q4. TE

Pipeline runs are queued but never start. What do you check?

Answer:

Queue without execution causes: (1) **No available agents** — all agents are busy or offline; check agent pool capacity. (2) **Concurrency limit reached** — free tier has 1 concurrent job; purchase parallel jobs or use self-hosted. (3) **Agent demands not met** — pipeline requires capabilities not present on available agents. (4) **Pipeline is disabled** — check pipeline settings. (5) **Branch filter mismatch** — trigger is configured but branch name doesn't match the pattern.

Q5. TE

A Docker build succeeds locally but fails in the pipeline with 'file not found'. What could cause this?

Answer:

Common causes: (1) **.dockerignore too aggressive** — required files are excluded. (2) **Working directory mismatch** — pipeline clones to a specific directory; Dockerfile context path may be wrong. (3) **Case sensitivity** — Linux agents are case-sensitive; Windows local build is not. (4) **Missing checkout step** — ensure source code is checked out before Docker build. (5) **Build context incorrect** — ensure Docker task buildContext is set correctly to include all required files.

Q6. ADVANCED

How do you troubleshoot a failing Kubernetes deployment in Azure Pipelines?

Answer:

K8s deployment troubleshooting: (1) **kubectl describe pod pod-name** — check events for image pull errors, OOM kills, readiness probe failures. (2) **kubectl logs pod-name --previous** — logs from crashed container. (3) **Image pull failure** — verify imagePullSecret for ACR, check if image exists with correct tag. (4) **OOMKilled** — increase memory limits. (5) **CrashLoopBackOff** — application startup failure; check application logs. (6) Check **kubectl get events -n namespace** for cluster-level issues.

Q7. ADVANCED

Terraform apply is failing with a state lock error. How do you resolve it?

Answer:

State lock resolution: (1) Check if another **terraform apply is actually running** in another pipeline — wait for it to complete. (2) If no active run exists, the lock is stale (previous run crashed). (3) Run **terraform force-unlock LOCK_ID** with the lock ID from the error message. (4) In Azure Storage backend, manually break the blob lease via Azure Portal or Azure CLI: **az storage blob lease break**. (5) Investigate why the previous run crashed — fix root cause before re-applying.

Q8. ADVANCED

Tests are passing in the pipeline but users report bugs in production. What's wrong with your testing strategy?

Answer:

Gap analysis: (1) **Test pyramid imbalance** — too many unit tests, insufficient integration/e2e tests. (2) **Test data quality** — tests use synthetic data not representative of production data. (3) **Environment parity** — staging differs from production (different DB version, missing services). (4) **Missing non-functional tests** — performance, security, accessibility not tested. (5) Remediation: add **contract tests** (Pact), improve production-like staging, implement **synthetic monitoring** in production, add real user monitoring.

Q9. TE

How do you handle flaky tests in a CI pipeline?

Answer:

Flaky test management: (1) **Identify flaky tests** — Azure DevOps Test Plans shows flaky test rate in pipeline analytics. (2) **Quarantine approach** — move confirmed flaky tests to a separate suite; run in parallel but don't block merge. (3) **Fix root causes** — timing issues (add proper waits), test isolation issues (clean state between tests), external dependency flakiness (mock or use dedicated test resources). (4) **Re-run on failure** — configure pipeline to retry failed tests once before failing the build.

Q10. TE

A YAML pipeline template is not loading correctly. How do you debug template issues?

Answer:

Template debugging: (1) Check **template file path and repository reference** — case-sensitive on Linux. (2) Ensure the template repository is added as a **resource** in the main pipeline with correct ref/branch. (3) Use **pipeline preview** (dry run) to validate YAML expansion without running. (4) Validate YAML syntax using **azure-pipelines YAML validator** in VS Code extension. (5) Check that **required parameters** are all passed. (6) Verify the pipeline has **access to the template repository**.

Q11. TE

An Azure Web App deployment succeeds in the pipeline but the application shows the old version. Why?

Answer:

Causes of stale deployment: (1) **CDN caching** — serving old static assets; force cache bust with versioned asset URLs. (2) **Wrong deployment slot** — deployed to staging, slot swap didn't happen. (3) **Deployment to wrong region** — multi-region app, traffic manager still routing to non-updated region. (4) **App Service not restarted** — add a restart step or enable restartAppOnFailure. (5) **Application-level caching** — in-memory cache not invalidated on deployment.

Q12. ADVANCED

How do you investigate high memory usage in an AKS pod causing OOMKill events?

Answer:

OOMKill investigation: (1) **kubectl top pod pod-name** — current memory usage vs limits. (2) Review **Container Insights** for memory trend over time. (3) **Heap dumps** — trigger via Application Insights Profiler or kubectl exec into pod. (4) **Memory leak vs spike** — gradual increase = leak; sudden spike = bursty workload. (5) Short-term: increase memory limit. (6) Long-term: profile application, fix memory leak, implement **HPA** to scale horizontally.

Q13. ADVANCED

A pipeline is succeeding but deploying the wrong version of the application. How do you track this down?

Answer:

Version tracking investigation: (1) Check pipeline **artifact version** — which build artifact was downloaded by the deploy stage. (2) Verify **artifact source** — is the deploy pipeline downloading from the correct upstream pipeline. (3) Check **image tag** — is 'latest' being used? Always use specific tags (git SHA or build ID). (4) Check **pipeline triggers** — did the deploy trigger from the right CI pipeline completion. (5) Compare **deployed image SHA** in AKS (kubectl describe pod) with expected SHA.

Q14. TE

How do you recover from an accidental git push that deleted a remote branch?

Answer:

Recovery: (1) If local copy exists: **git push origin branch-name** to restore. (2) If no local copy: check other team members' machines — anyone with the branch locally can restore it. (3) In Azure Repos: check **Pushes history** — deleted branch may appear with commits. (4) Use **git reflog** on a machine that had the branch to find the lost commit SHA. (5) Enable **Branch policies** on critical branches — policies prevent force-push and deletion.

Q15. ADVANCED

How do you detect and resolve configuration drift in infrastructure managed by Terraform?

Answer:

Drift detection and resolution: (1) Run **terraform plan** regularly (scheduled pipeline) to detect differences between state and actual infrastructure. (2) If drift is due to **authorized manual changes**: update the code to reflect the change or run terraform import. (3) If drift is due to **unauthorized changes**: run terraform apply to restore desired state; investigate who made the change via Azure Activity Logs. (4) Enable **Azure Policy** to prevent unauthorized resource modifications.
